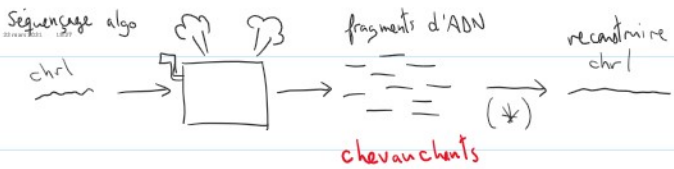
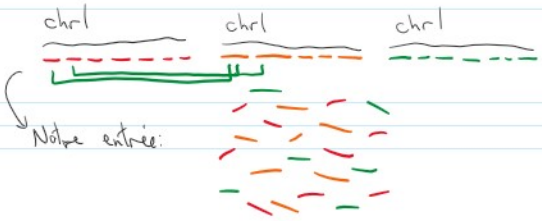




Problème:

<u>Entrée</u>: fragments $r_1, r_2, \dots, r_n$ proviennent de fragmentations différentes
chevauchants
appelés reads (lectures)

<u>Sortie</u>: séquence originale

- Problème de la + courte super-séquence

<u>Entrée</u>: $r_1, r_2, \dots, r_n$

<u>Sortie</u>: plus courte séquence S tq. chaque r_i est contenu dans S.

ex: r_1 = ACCG r_2 = CGCT r_3 = GCTA r_4 = GACC

S = GACCGCTA

r_4 r_1 r_2 r_3

- Ce pb est NP-complet. (problème pas d'algo poly(n))
- Méthode de résolution classique
  - réduction au pb du commis voyageur (TSP)

- on définit overlap(r_i, r_j) = taille du + long suffixe de r_i qui est un préfixe de r_j

ex: r_i = ACGGT r_j = GGTTTT

overlap(r_i, r_j) = 3

ACGGT
GGTTTT → ACGGTTTT

- construire un graphe $G = (V, E)$ orienté

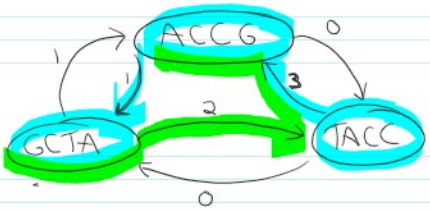
sommets → V E → arêtes (forme (u,v))

V: sommets: $\{r_1, r_2, \dots, r_n\}$

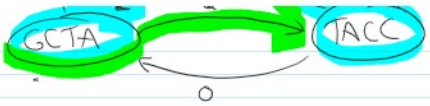
E: arête = $\{(r_i, r_j) : i, j \in \{1, \dots, n\}, i \neq j\}$

poids sur (r_i, r_j) = overlap(r_i, r_j)

r_1 = ACCG r_2 = GCTA r_3 = TACC



<u>But</u>: trouver un chemin qui passe exactement 1 fois par sommet et qui max. la somme des poids des arêtes.



poids des arêtes.

= TACC GCTA (algo glouton)
= GCTACCG (mieux)

└ Algo glouton :
- P = ∅
- commencer par l'arête (u,v) de poids max
- P.append(u)
- P.append(v)
- tant que |P| ≤ n
  - w = P.last()
  - trouvez z ∉ P qui max. le poids de (w, z)
  - P.append(z)
- return P

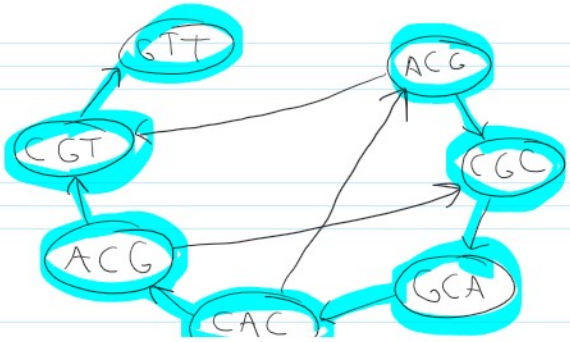
<u>Thm</u> : algo-glouton est une 4-approximation
sol glouton ≤ 4 · optimale

<u>Conjecture</u> : glouton ≤ 2 · optimale

Décomposition en k-mer
- k-mer : séq. de longueur k
- Lister tous les k-mer de chaque read (read long n, n-k+1 k-mers)

  r := ACCGT k = 3
  ↳ ACC CCG CGT
- Si chaque k-mer de la séq. originale S est présent, on devrait pouvoir reconstruire S avec seulement les k-mers.
- On va construire un graphe G = (V, E) où
  V = tous les k-mers (version avec ou sans répétition)
  E = { (X, Y) : X = aX' et Y = X'b , a, b ∈ Σ }
  chevauchement de taille k-1

  ex: k-mers ACG CGC GCA CAC ACG CGT GTT (avec répét.)



Graphe des k-mers
Idée : 2 k-mer consécut. fo = seq taille 4 (k+1)
ex: ACG → CGC = ACGC

<u>But</u> : trouver un chemin qui passe par chaque sommet 1 fois

[redacted] : ACGCACGTT

— chaque sommet 1 fois

- Quand même NP-complet
- Autre formalisation : #min de chemins disjoints pour couvrir chaque sommet

---

Graphes de Bruijn

- Autre modélisation
- Au lieu que sommet = k-mer
  → plutôt arête = k-mer
- Graphe $G = (V, E)$

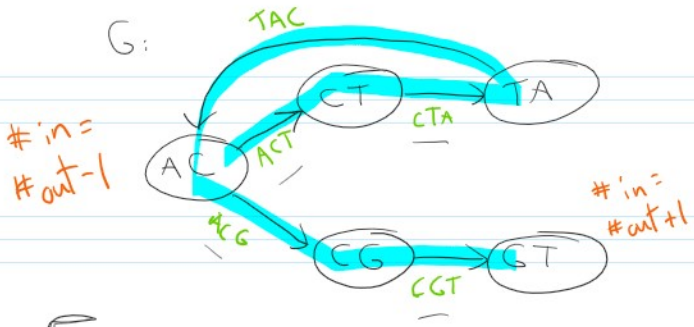
V = Tous les k-1 mers possibles qui apparaissent dans nos reads (sans répétition)

$E = \{ (X, Y) : \exists$ k-mer Z dans nos reads tel que $Z = aX$ et $Z = Yb$, $a, b \in \Sigma \}$

(avec répétition si multiples k-mers qui lient X et Y)

ex : Reads = CGT ACG CTA ACT TAC k=3

G :



arête = mes k-mers

<u>But</u> : trouver un chemin qui contient chaque arête 1 fois.

On appelle ceci un chemin Eulérien.

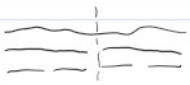
- On peut déterminer en <u>temps polynomial</u> s'il $\exists$ un chemin avec chaque arête 1 fois.

[redacted] : ACTACGT

#in = nb voisin entr.
#out = nb voisins sort.

- <u>Thm</u> : $\exists$ un chemin Eulérien si et seulement si
  - il y a ≤ 1 sommet tel que #in = #out - 1 ← départ
  - il y a ≤ 1 sommet tel que #in = #out + 1 ← arrivée
  - #in = #out pour tout autre sommet ← intermédiaire

<u>Supposition</u> <u>Réalité</u>



Supposition	Réalité
• chaque k-mer de la séq. originale est dans nos reads	• un k-mer peut être absent
• pas d'erreur dans nos reads	• taux d'erreur variables sur nucl.
• solution unique	• plein de sds └ segments fortement répétés .... ACAAAACAAAA ...

Comment stocker un génome séquencé

└ la méthode de stockage doit supporter la recherche de sous-séq.

└ arbre de suffixe > 25 Gb humain

└ STAR: tab. de suffixe > 15 Gb
└ en RAM = beaucoup

└ Transformée de Burrows-Wheeler

- permet la compression (< 10 Gb)
- soit une séq. S à stocker ($n = |S|$)
  └ symbole de fin \$ < tout autre symb.
- traiter S comme une séquence circulaire
  └ n mots possibles de taille n (rotations)
- générer les n rotations
- trier ces rotations en ordre alphabétique (\$ < tout)
- BWT(S) = dernière colonne de la matrice des rotations

  dans l'exemple: BWT(S) = D\$ABC

S = ABCD\$

circulaire

mots possibles:

\$ABCD
ABCD\$
BCD\$A
CD\$AB
D\$ABC

tab 2D = matrice des rotations

exemple: BANANA\$ ⟶

```
$BANANA
A$BANAN
ANA$BAN
ANANA$B
BANANA$
NA$BANA
NANA$BA
```

BWT(BANANA\$) = ANNB\$AA

Avantages: • BWT(S) est facilement compressible

- La dernière colonne a souvent beaucoup de caractères identiques consécutifs

  ANNB\$AA

  Encodage "run-length": chaîne de k N → kN

  ...NNNNNNN... → ...7N

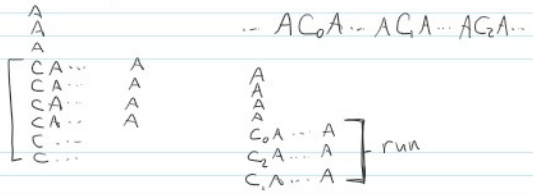
  ex: AAAANNNNNCCAAAAAGGGG ⟶ 4A5N2C5A4A
  transformée finale

  Pourquoi? Disons qu'on a beaucoup d'occurrences d'un même triplet, par ex. ACA

ex. AAAANNNNCCANAAAGGGG → $\underbrace{\text{AANNACAAA}}_{\text{transformée finale}}$

Pourquoi? Disons qu'on a beaucoup d'occurrences d'un même triplet, par ex. ACA

Alors bcp de rotations débutant par C finissent par A

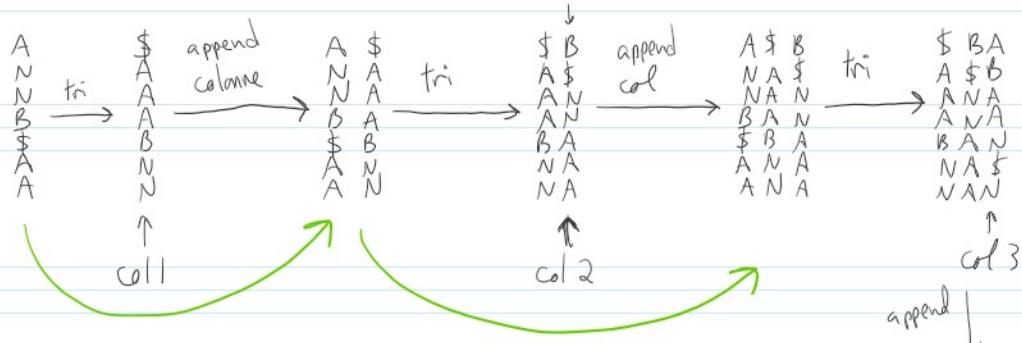


- Permet la recherche d'occ. d'un mot P dans S (avec $O(|BWT(S)|)$ données auxiliaires)
- Permet de reconstruire la séq. originale (réversibilité)

ANNB$AA → → BANANA$

Comment reconstruire S à partir de BWT(S)

└ trier les car. de BWT(S), on obtient la colonne 1 de la matrice



matrice complète
mot orig reconstructible
à partir de la rangée 1

→

$BANANA
A$BANAN
ANA$BAN
ANANA$B
BANANA$
NA$BANA
NANA$BA